



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа

«Решение баллистической задачи. Модель Ньютона.»
по курсу «Моделирование»

Студент: Аригин М.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Постановка задачи

Рассматривается задача о движении тела, брошенного под углом к горизонту. Тело массой m запускается с начальной скоростью v_0 под углом α к горизонту и движется в вертикальной плоскости под действием силы тяжести. В зависимости от учета сопротивления воздуха различают две модели: модель Галилея (без учета сопротивления) и модель Ньютона (с учетом сопротивления воздуха, пропорционального квадрату скорости).

Цель работы — численное решение данной задачи методом Рунге-Кутты 4-го порядка для обеих моделей, сравнение траекторий и анализ влияния сопротивления воздуха на движение тела.

Входные параметры:

- $g = 9.81 \text{ м/с}^2$ — ускорение свободного падения
- $v_0 = 60.0 \text{ м/с}$ — начальная скорость
- $\alpha = 43$ — угол бросания
- $C = 0.15$ — коэффициент сопротивления
- $\rho_{\text{air}} = 1.225 \text{ кг/м}^3$ — плотность воздуха
- $r_{\text{core}} = 0.04 \text{ м}$ — радиус ядра
- $\rho_{\text{core}} = 7300.0 \text{ кг/м}^3$ — плотность олова

Выходные параметры:

- $S = \pi r_{\text{core}}^2$ — площадь поперечного сечения
- $V_{\text{core}} = \frac{4}{3}\pi r_{\text{core}}^3$ — объем ядра
- $m = \rho_{\text{core}} \cdot V_{\text{core}}$ — масса ядра
- $\beta = \frac{C\rho_{\text{air}}S}{2}$ — коэффициент сопротивления
- R — дальность полета

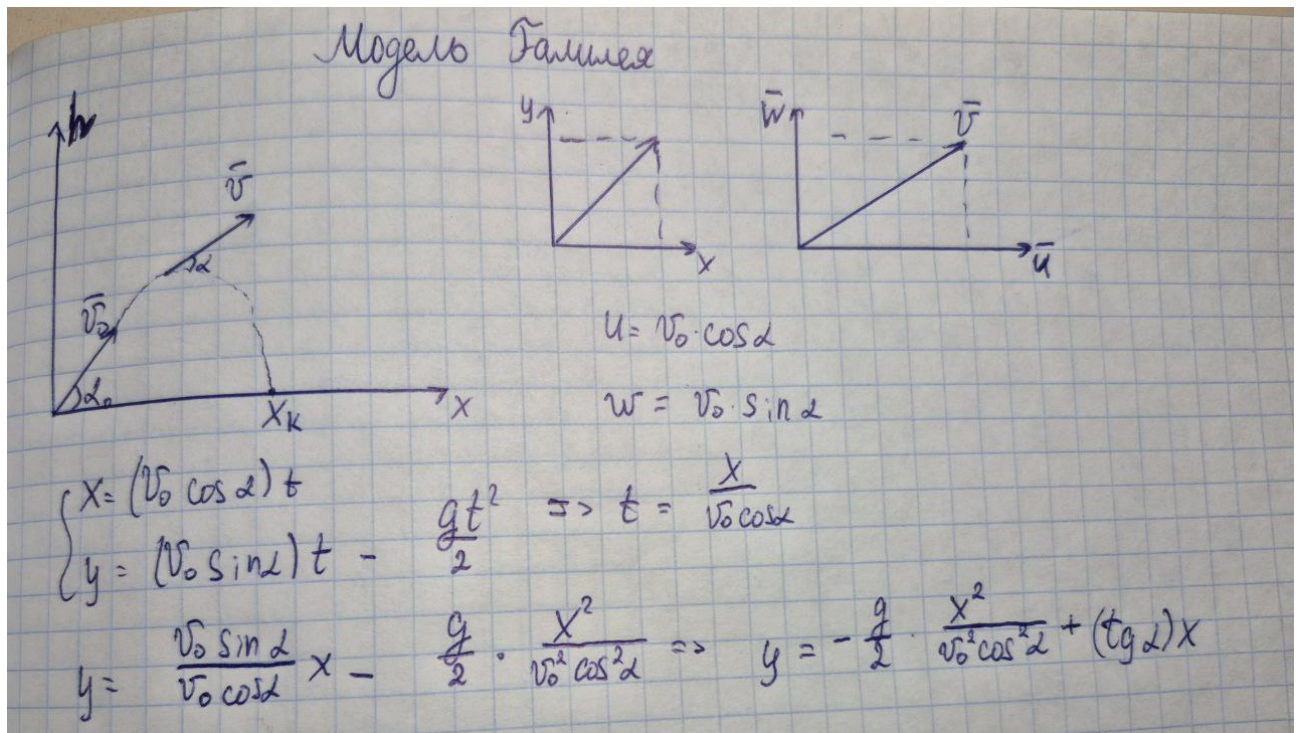
2 Построение математической модели

Рассматривается движение тела в вертикальной плоскости (x, y) при следующих допущениях:

- ускорение свободного падения g постоянно;
- плотность воздуха ρ_{air} постоянна;
- тело рассматривается как материальная точка массой m ;
- сопротивление воздуха направлено против скорости и пропорционально квадрату модуля скорости.

Введем переменные состояния: координаты (x, y) и проекции скорости (u, w) .

2.1 Модель Галилея



В модели Галилея сопротивление воздуха не учитывается. Система дифференциальных уравнений имеет вид:

$$\begin{cases} \frac{dx}{dt} = u, \\ \frac{dy}{dt} = w, \\ \frac{du}{dt} = 0, \\ \frac{dw}{dt} = -g. \end{cases}$$

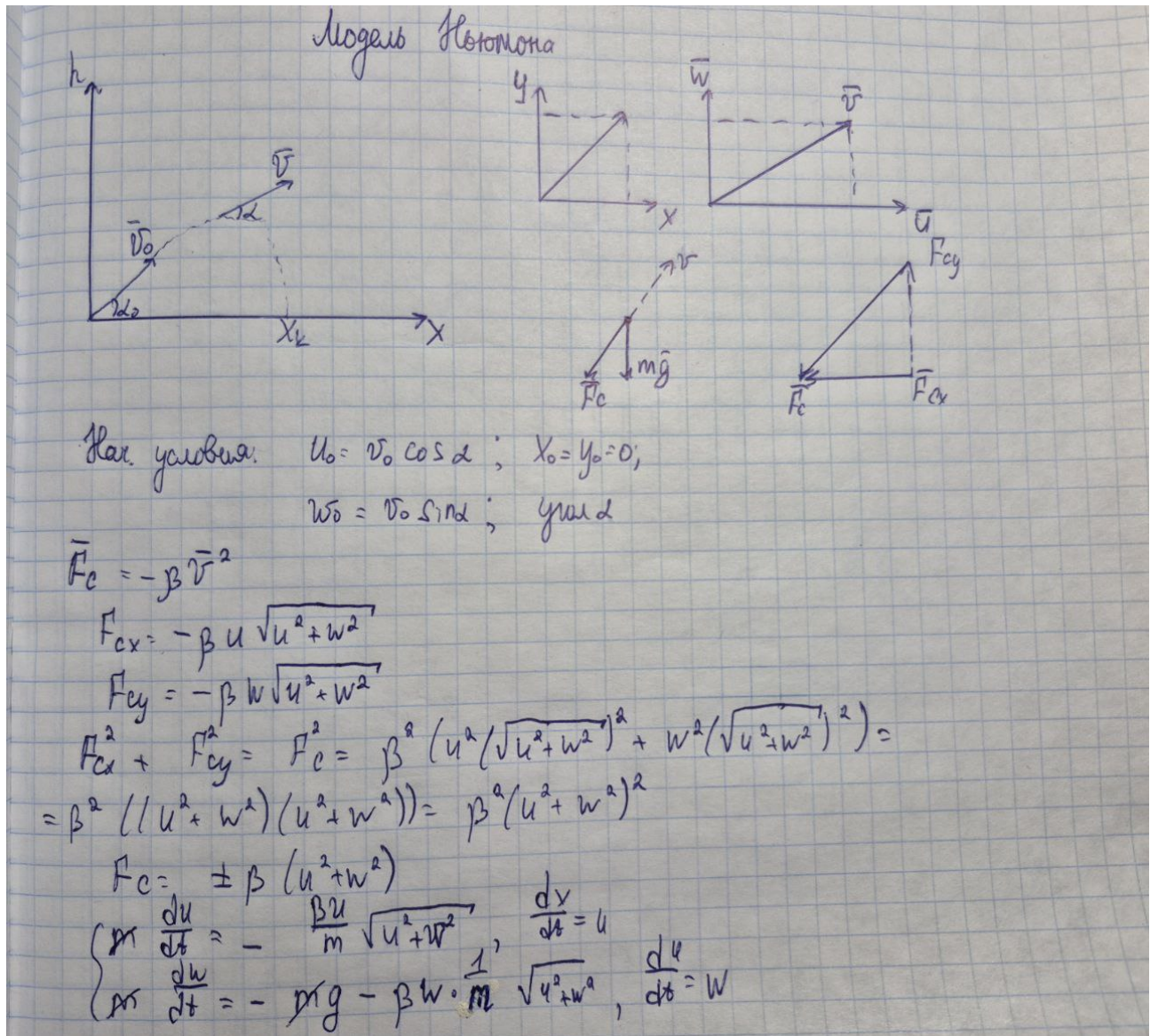
Начальные условия:

$$x(0) = 0, \quad y(0) = 0, \quad u(0) = v_0 \cos \alpha, \quad w(0) = v_0 \sin \alpha.$$

В данной модели траектория движения является параболой. Аналитическое решение:

$$x(t) = v_0 \cos \alpha \cdot t, \quad y(t) = v_0 \sin \alpha \cdot t - \frac{g}{2} t^2.$$

2.2 Модель Ньютона



В модели Ньютона учитывается сопротивление воздуха, пропорциональное квадрату скорости. Сила сопротивления задается как

$$\vec{F}_d = -\beta V \vec{v}, \quad V = \sqrt{u^2 + w^2},$$

где $\beta = \frac{C \rho_{\text{air}} S}{2}$.

Система дифференциальных уравнений:

$$\begin{cases} \frac{dx}{dt} = u, \\ \frac{dy}{dt} = w, \\ \frac{du}{dt} = -\frac{\beta}{m}uV, \\ \frac{dw}{dt} = -g - \frac{\beta}{m}wV. \end{cases}$$

Начальные условия:

$$x(0) = 0, \quad y(0) = 0, \quad u(0) = v_0 \cos \alpha, \quad w(0) = v_0 \sin \alpha.$$

Связь параметров с геометрией и материалом для сферического ядра радиуса r_{core} :

$$S = \pi r_{\text{core}}^2, \quad V_{\text{core}} = \frac{4}{3}\pi r_{\text{core}}^3, \quad m = \rho_{\text{core}} V_{\text{core}}, \quad \beta = \frac{C \rho_{\text{air}} S}{2}.$$

В данной модели траектория движения не является параболой из-за нелинейного члена сопротивления.

3 Решение вычислительной задачи

Численное решение строится в виде последовательности подзадач.

3.1 Постановка задачи Коши

Система ОДУ первого порядка с начальными условиями представляет собой корректную задачу Коши. Для модели Ньютона правая часть непрерывна. При $V \rightarrow 0$ возможна численная неустойчивость (деление на очень малые значения), поэтому в реализации вводится проверка $V < \varepsilon$.

3.2 Выбор временного интервала

Для модели Галилея высота

$$y(t) = v_0 \sin \alpha \cdot t - \frac{g}{2} t^2$$

обращается в ноль при двух значениях времени: $t_1 = 0$ (вырожденное решение в начальной точке) и

$$t_2 = \frac{2v_0 \sin \alpha}{g}$$

(время полета). Для численного расчета берется невырожденный корень t_2 и небольшой запас по времени.

3.3 Численное интегрирование

Используется метод Рунге–Кутты 4-го порядка (RK4):

$$\begin{aligned} k_1 &= \Delta t f(t_n, y_n), \\ k_2 &= \Delta t f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{k_1}{2}\right), \\ k_3 &= \Delta t f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{k_2}{2}\right), \\ k_4 &= \Delta t f(t_n + \Delta t, y_n + k_3), \\ y_{n+1} &= y_n + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}. \end{aligned}$$

3.4 Критерий окончания полета

Интегрирование останавливается при $y < 0$ (касание земли). Дальность полета оценивается как $R = \max x(t)$ на интервале, где $y(t) \geq 0$.

4 Анализ моделей

Сравнение двух моделей позволяет оценить влияние сопротивления воздуха на траекторию движения тела.

В модели Галилея траектория параболическая и дальность максимальна при заданных v_0 и α . В модели Ньютона дальность уменьшается, а траектория ста-

новится непараболической из-за нелинейного члена сопротивления. Влияние сопротивления воздуха усиливается при росте v_0 , увеличении r_{core} (рост S) и уменьшении плотности материала (уменьшение m при фиксированном r_{core}).

Ограничения моделей: постоянное g , отсутствие ветра, постоянные C и ρ_{air} , отсутствие подъемной силы и вращения тела. Эти допущения упрощают расчет, но ограничивают применимость результатов.

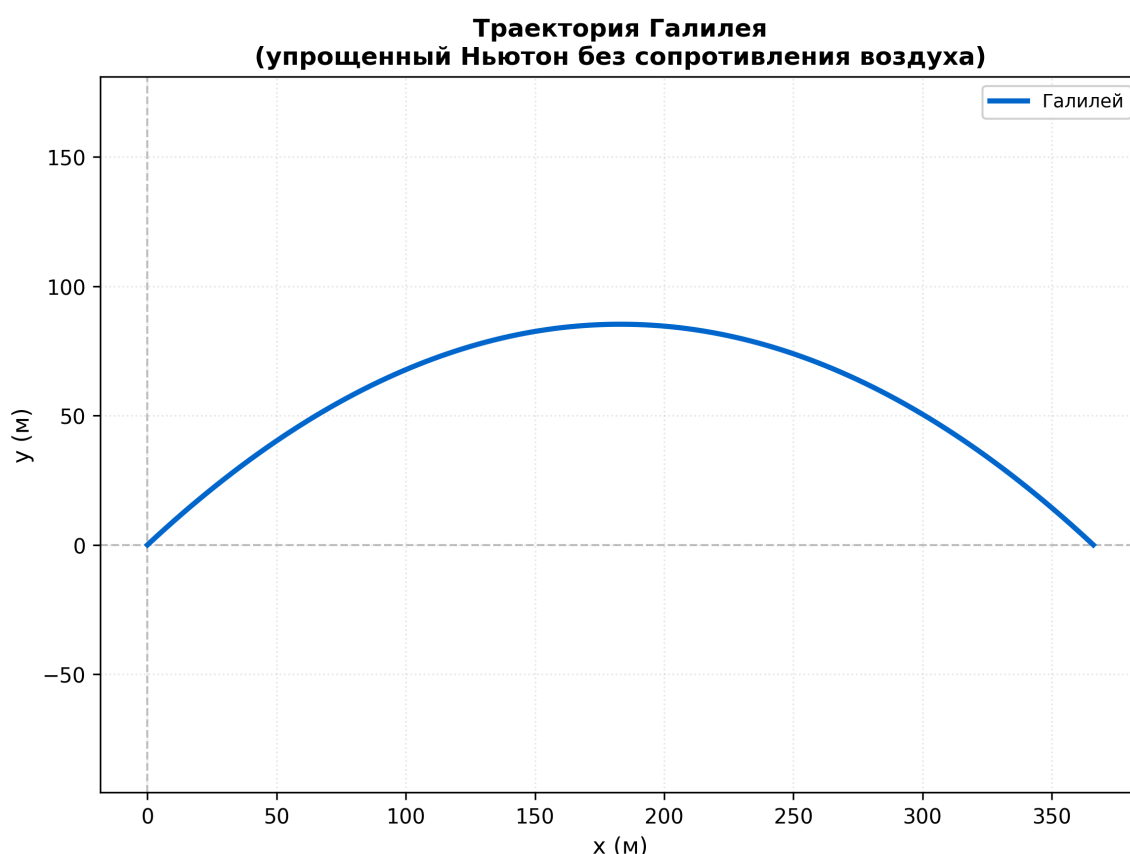


Рисунок 1 Траектория движения тела в модели Галилея

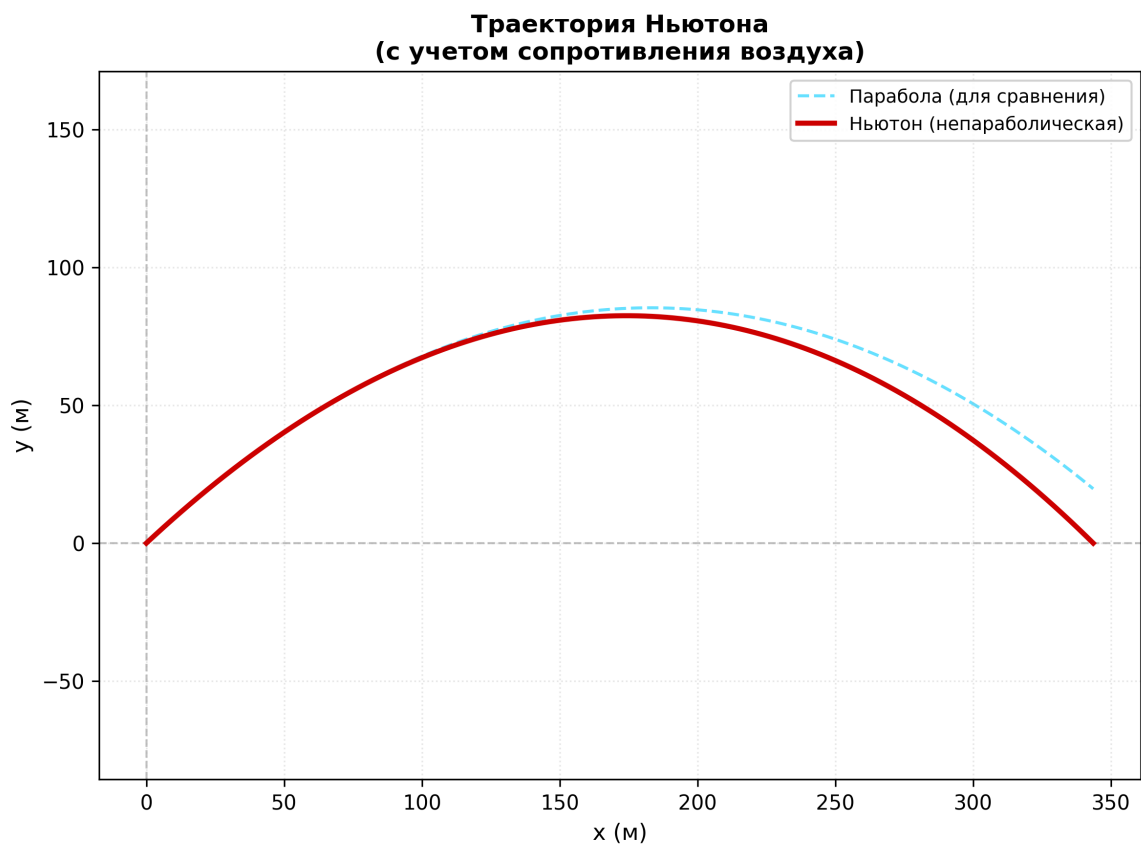


Рисунок 2 Траектория движения тела в модели Ньютона

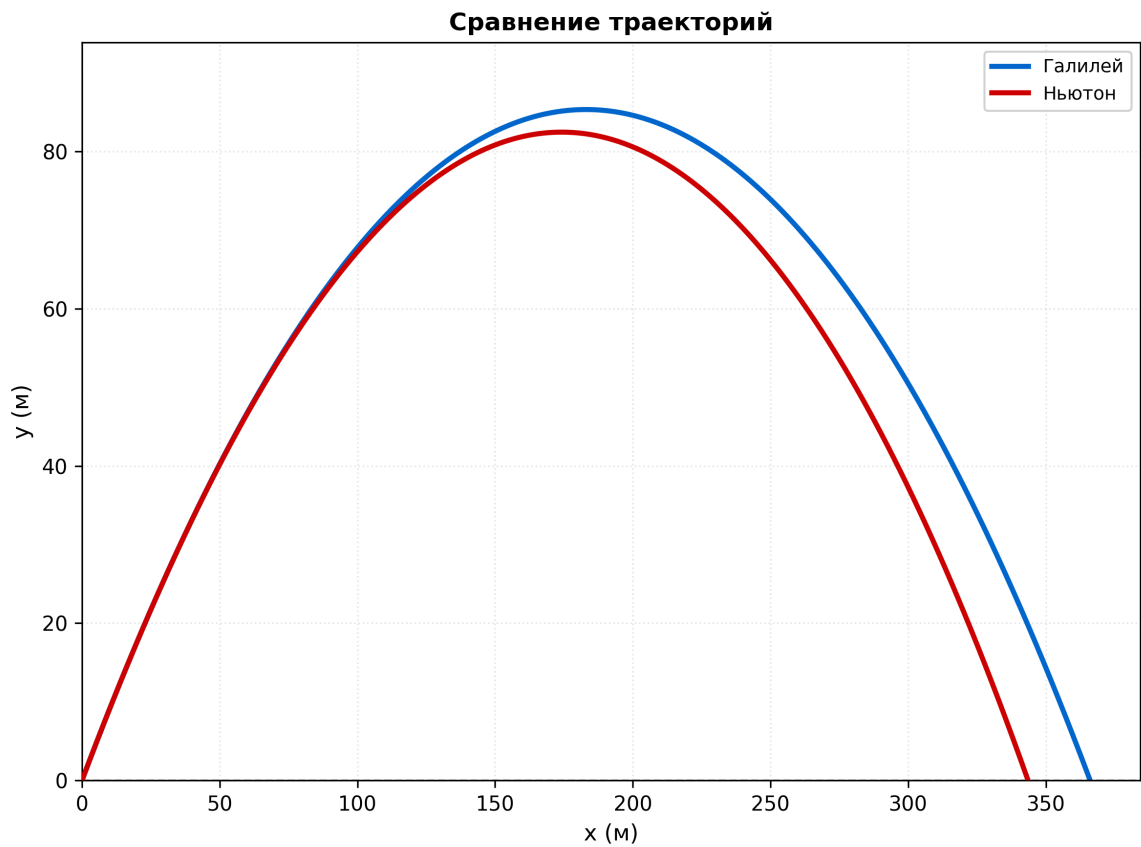


Рисунок 3 Сравнение траекторий моделей Галилея и Ньютона

5 Практическая реализация

Listing 1: Модель Галилея и Ньютона

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 class RungeKuttaSolver:
6     def __init__(self, dt=0.01):
7         self.dt = dt
8
9     def rk4_step(self, f, t, y):
10         k1 = self.dt * f(t, y)
11         k2 = self.dt * f(t + self.dt/2, y + k1/2)
12         k3 = self.dt * f(t + self.dt/2, y + k2/2)
13         k4 = self.dt * f(t + self.dt, y + k3)
14
15         return y + (k1 + 2*k2 + 2*k3 + k4) / 6
16
17     def solve(self, f, y0, t_span):
18         y = np.zeros((len(t_span), len(y0)))
19         y[0] = y0
20
21         for i in range(1, len(t_span)):
22             y[i] = self.rk4_step(f, t_span[i-1], y[i-1])
23
24         return y
25
26
27 class GalileiModel:
28     def __init__(self, g=9.81, v0=1.0, alpha=45.0):
29         self.g = g
30         self.v0 = v0
31         self.alpha = np.deg2rad(alpha)
32
33     def solve_numerically(self, t_span, dt=0.01):
34         t_flight = 2.0 * self.v0 * np.sin(self.alpha) / self.g
35
36
37         if len(t_span) > 0 and t_span[-1] < t_flight * 1.5:
38             t_max = t_flight * 1.5
39             t_span = np.arange(0, t_max, dt)
40
41         x0 = 0.0
42         y0 = 0.0
43         u0 = self.v0 * np.cos(self.alpha)
44         w0 = self.v0 * np.sin(self.alpha)
```

```

44
45 y_init = np.array([x0, y0, u0, w0])
46
47 def f_galilei(t, y):
48     x, y_pos, u, w = y
49     return np.array([u, w, 0, -self.g])
50
51 solver = RungeKuttaSolver(dt=dt)
52 solution = solver.solve(f_galilei, y_init, t_span)
53
54 x = solution[:, 0]
55 y = solution[:, 1]
56
57 mask = y >= 0
58 if not np.all(mask):
59     first_negative = np.where(~mask)[0]
60     if len(first_negative) > 0:
61         mask = np.arange(len(y)) < first_negative[0]
62     else:
63         last_positive = np.where(y >= 0)[0]
64         if len(last_positive) > 0:
65             mask = np.arange(len(y)) <= last_positive[-1]
66
67     return x[mask], y[mask]
68
69
70 class NewtonModel:
71     def __init__(self, g=9.81, v0=1.0, alpha=45.0, m=None, r_core=None, rho_core=None,
72                 beta=None, C=0.15, rho_air=1.225, S=None):
73         self.g = g
74         self.v0 = v0
75         self.alpha = np.deg2rad(alpha)
76
77         if m is None:
78             V_core = (4.0 / 3.0) * np.pi * r_core**3
79             self.m = rho_core * V_core
80             self.r_core = r_core
81             self.rho_core = rho_core
82         else:
83             self.m = m
84             self.r_core = r_core
85             self.rho_core = rho_core
86
87         if S is None:
88             if r_core is not None:
89                 self.S = np.pi * r_core**2
90         else:

```

```

91         self.S = S
92
93     if beta is None:
94         self.beta = C * rho_air * self.S / 2.0
95     else:
96         self.beta = beta
97
98     def solve(self, t_span, dt=0.01):
99         x0 = 0.0
100        y0 = 0.0
101        u0 = self.v0 * np.cos(self.alpha)
102        w0 = self.v0 * np.sin(self.alpha)
103
104        y_init = np.array([x0, y0, u0, w0])
105
106        def f_newton(t, y):
107            x, y_pos, u, w = y
108            V = np.sqrt(u**2 + w**2)
109
110            if V < 1e-10:
111                du_dt = 0
112                dw_dt = -self.g
113            else:
114                du_dt = -self.beta * u * V / self.m
115                dw_dt = -self.g - self.beta * w * V / self.m
116
117            return np.array([u, w, du_dt, dw_dt])
118
119        solver = RungeKuttaSolver(dt=dt)
120        solution = solver.solve(f_newton, y_init, t_span)
121
122        x = solution[:, 0]
123        y = solution[:, 1]
124        u = solution[:, 2]
125        w = solution[:, 3]
126
127        mask = y >= 0
128        return x[mask], y[mask], u[mask], w[mask]
129
130
131     def plot_trajectories(g=9.81, v0=30.0, alpha=45.0, C=0.5, rho_air=1.225,
132                          S=None, r_core=0.04, rho_core=200.0, diff_range=None, dt=0.001,
133                          filename='trajectory_comparison.png'):
134         if S is None:
135             S = np.pi * r_core**2
136
137         t_flight_galilei = 2.0 * v0 * np.sin(np.deg2rad(alpha)) / g

```

```

138 t_max = max(5.0, t_flight_galilei * 1.5)
139 t_span = np.arange(0, t_max, dt)
140
141 galilei = GalileiModel(g=g, v0=v0, alpha=alpha)
142 x_gal, y_gal = galilei.solve_numerically(t_span, dt=dt)
143
144 newton = NewtonModel(g=g, v0=v0, alpha=alpha, r_core=r_core, rho_core=rho_core,
145                      C=C, rho_air=rho_air)
146 x_newt, y_newt, u_newt, w_newt = newton.solve(t_span, dt=dt)
147
148 plt.figure(figsize=(8, 6))
149 if len(x_gal) > 0 and len(y_gal) > 0:
150     plt.plot(x_gal, y_gal, color='#0066CC', linestyle='-', linewidth=2.5,
151             label='Галилей', zorder=2)
152     x_max_gal = np.max(x_gal)
153     y_max_gal = np.max(y_gal)
154     plt.xlim(-0.005, x_max_gal * 1.05)
155     plt.ylim(-0.005, y_max_gal * 1.1)
156
157     plt.axhline(y=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
158     plt.axvline(x=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
159
160 plt.xlabel('x (м)', fontsize=11)
161 plt.ylabel('y (м)', fontsize=11)
162 plt.title('Траектория Галилея\n(упрощенный Ньютон без сопротивления воздуха)',
163          fontsize=12, fontweight='bold')
164 plt.legend(fontsize=9, loc='upper right', framealpha=0.9)
165 plt.grid(True, alpha=0.3, linestyle=':')
166 plt.axis('equal')
167 plt.tight_layout()
168 plt.savefig('trajectory_galilei.png', dpi=300, bbox_inches='tight')
169 plt.close()
170
171 plt.figure(figsize=(8, 6))
172 if len(x_newt) > 0 and len(y_newt) > 0:
173     x_parabola = np.linspace(0, np.max(x_newt), 1000)
174     y_parabola = -g / (2 * v0**2 * np.cos(np.deg2rad(alpha))**2) * x_parabola**2 +
175                 ↪ np.tan(np.deg2rad(alpha)) * x_parabola
176     y_parabola = y_parabola[y_parabola >= 0]
177     x_parabola = x_parabola[:len(y_parabola)]
178
179     plt.plot(x_parabola, y_parabola, color='#00CCFF', linestyle='--', linewidth=1.5,
180             label='Парабола (для сравнения)', zorder=1, alpha=0.6)
181     plt.plot(x_newt, y_newt, color='#CC0000', linestyle='-', linewidth=2.5,
182             label='Ньютон (непараболическая)', zorder=2)
183     x_max_newt = np.max(x_newt)
184     y_max_newt = np.max(y_newt)

```

```

184     plt.xlim(-0.005, x_max_newt * 1.05)
185     plt.ylim(-0.005, y_max_newt * 1.1)
186
187     plt.axhline(y=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
188     plt.axvline(x=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
189
190     plt.xlabel('x (м)', fontsize=11)
191     plt.ylabel('y (м)', fontsize=11)
192     plt.title('Траектория Ньютона\n(с учетом сопротивления воздуха)',
193             fontsize=12, fontweight='bold')
194     plt.legend(fontsize=9, loc='upper right', framealpha=0.9)
195     plt.grid(True, alpha=0.3, linestyle=':')
196     plt.axis('equal')
197     plt.tight_layout()
198     plt.savefig('trajectory_newton.png', dpi=300, bbox_inches='tight')
199     plt.close()
200
201     plt.figure(figsize=(8, 6))
202     if len(x_gal) > 0 and len(x_newt) > 0:
203         if diff_range is None:
204             range_gal = np.max(x_gal)
205             range_newt = np.max(x_newt)
206             diff_range = range_gal - range_newt
207         else:
208             range_gal = np.max(x_gal)
209             range_newt = np.max(x_newt)
210
211         t_gal = np.arange(0, len(x_gal)) * dt
212         t_newt = np.arange(0, len(x_newt)) * dt
213
214         t_min = 0
215         t_max_common = min(t_gal[-1] if len(t_gal) > 0 else 0, t_newt[-1] if len(t_newt) > 0 else 0)
216         t_common = np.linspace(t_min, t_max_common, 1000)
217
218         x_gal_interp = np.interp(t_common, t_gal, x_gal)
219         x_newt_interp = np.interp(t_common, t_newt, x_newt)
220
221         diff_x = x_gal_interp - x_newt_interp
222
223         plt.plot(x_gal_interp, diff_x, color='purple', linestyle='-', linewidth=2.5,
224                 label='Разница по x (Галилей - Ньютон)', zorder=2)
225         plt.axhline(y=0, color='black', linestyle='--', linewidth=1, alpha=0.5)
226
227         plt.xlabel('x (м)', fontsize=11)
228         plt.ylabel('Разница x (м)', fontsize=11)
229         plt.title('Погрешность по дальности (x)', fontsize=12, fontweight='bold')
230         plt.legend(fontsize=9, loc='best', framealpha=0.9)

```

```

231     plt.grid(True, alpha=0.3, linestyle=':')
232
233 plt.tight_layout()
234 plt.savefig('trajectory_error.png', dpi=300, bbox_inches='tight')
235 plt.close()
236
237 plt.figure(figsize=(8, 6))
238 if len(x_gal) > 0 and len(x_newt) > 0:
239     plt.plot(x_gal, y_gal, color='#0066CC', linestyle='--', linewidth=2.5,
240             label='Галилей', zorder=2)
241     plt.plot(x_newt, y_newt, color='#CC0000', linestyle='--', linewidth=2.5,
242             label='Ньютон', zorder=3)
243     plt.axhline(y=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
244     plt.axvline(x=0, color='gray', linestyle='--', linewidth=1, alpha=0.5, zorder=0)
245
246     x_max = max(np.max(x_gal), np.max(x_newt))
247     y_max = max(np.max(y_gal), np.max(y_newt))
248     plt.xlim(-0.005, x_max * 1.05)
249     plt.ylim(-0.005, y_max * 1.1)
250
251     plt.xlabel('x (м)', fontsize=11)
252     plt.ylabel('y (м)', fontsize=11)
253     plt.title('Сравнение траекторий', fontsize=12, fontweight='bold')
254     plt.legend(fontsize=9, loc='upper right', framealpha=0.9)
255     plt.grid(True, alpha=0.3, linestyle=':')
256
257 plt.tight_layout()
258 plt.savefig('trajectory_comparison.png', dpi=300, bbox_inches='tight')
259 plt.close()
260
261
262 if __name__ == "__main__":
263     g = 9.81
264     v0 = 60.0
265     alpha = 43.0
266     C = 0.15
267     rho_air = 1.225
268     r_core = 0.04
269     rho_core = 7300.0
270
271     S = np.pi * r_core**2
272     V_core = (4.0 / 3.0) * np.pi * r_core**3
273     m = rho_core * V_core
274     beta = C * rho_air * S / 2.0
275
276     t_flight_galilei = 2.0 * v0 * np.sin(np.deg2rad(alpha)) / g
277     t_max = max(1.0, t_flight_galilei * 1.5)

```

```

278 dt = 0.001
279 t_span = np.arange(0, t_max, dt)
280
281 print("=" * 60)
282 print("ВХОДНЫЕ ПАРАМЕТРЫ")
283 print("=" * 60)
284 print(f" g = {g} m/s^2")
285 print(f" v0 = {v0} m/s")
286 print(f" alpha = {alpha} grad")
287 print(f" C = {C}")
288 print(f" rho_air = {rho_air} kg/m^3")
289 print(f" S = {S} m^2 ({S*100:.1f} dm^2 = {S*10000:.0f} cm^2)")
290 print(f" rho_core = {rho_core} kg/m^3")
291
292 print("\n" + "=" * 60)
293 print("ВЫЧИСЛЯЕМЫЕ ПАРАМЕТРЫ")
294 print("=" * 60)
295 print(f" r_core = sqrt(S/pi) = {r_core:.6f} m")
296 print(f" V_core = (4/3)*pi*r^3 = {V_core:.6f} m^3")
297 print(f" m = rho_core * V_core = {m:.6f} kg")
298 print(f" beta = C*rho_air*S/2 = {beta:.6f}")
299 print(f" t_flight (Galilei) = 2*v0*sin(alpha)/g = {t_flight_galilei:.4f} s")
300 print(f" t_max = {t_max:.4f} s")
301 print(f" dt = {dt} s")
302
303 print("\n" + "=" * 60)
304 print("Модель Галилея (упрощенный Ньютон без сопротивления воздуха)")
305 print("=" * 60)
306 galilei = GalileiModel(g=g, v0=v0, alpha=alpha)
307 x_gal, y_gal = galilei.solve_numerically(t_span, dt=dt)
308 range_gal = np.max(x_gal)
309 print(f"Дальность полёта: {range_gal:.4f} m")
310
311 print("\n" + "=" * 60)
312 print("Модель Ньютона (с учетом сопротивления воздуха)")
313 print("=" * 60)
314 newton = NewtonModel(g=g, v0=v0, alpha=alpha, r_core=r_core, rho_core=rho_core,
315                      C=C, rho_air=rho_air)
316 x_newt, y_newt, u_newt, w_newt = newton.solve(t_span, dt=dt)
317 range_newt = np.max(x_newt)
318 print(f"Дальность полёта: {range_newt:.4f} m")
319
320 print("\n" + "=" * 60)
321 print("СРАВНЕНИЕ ДАЛЬНОСТИ ПОЛЁТА")
322 print("=" * 60)
323 diff_range = range_gal - range_newt
324

```

```

325     print(f"Разница в дальности: {diff_range:.6f} m ({diff_range*1000:.3f} mm)")
326
327     relative_error = abs(diff_range / range_gal) * 100
328     print(f"Относительная погрешность: {relative_error:.6f} %")
329
330
331     plot_trajectories(g=g, v0=v0, alpha=alpha, C=C, rho_air=rho_air,
332                     S=S, r_core=r_core, rho_core=rho_core,
333                     diff_range=diff_range, dt=dt)
334

```

6 Результаты

```

=====
Модель Галилея (упрощенный Ньютон без сопротивления воздуха)
=====
Дальность полёта: 366.0572 m

=====
Модель Ньютона (с учетом сопротивления воздуха)
=====
Дальность полёта: 343.5397 m

=====
СРАВНЕНИЕ ДАЛЬНОСТИ ПОЛЁТА
=====
Разница в дальности: 22.517482 m (22517.482 mm)
Относительная погрешность: 6.151357 %

```

В результате выполнения программы получены следующие результаты:

- **Дальность полета методом Галилея:** вычисляется численно методом Рунге-Кутты 4-го порядка
- **Дальность полета методом Ньютона:** вычисляется численно с учетом сопротивления воздуха

- **Разница в дальности:** показывает влияние сопротивления воздуха на траекторию
- **Графики траекторий:** построены отдельные графики для каждой модели и график сравнения

7 Выводы

- Метод Галилея дает параболическую траекторию, что соответствует классической теории движения тела под действием силы тяжести без учета сопротивления воздуха.
- Метод Ньютона с учетом сопротивления воздуха показывает, что траектория *не является параболой*. Сопротивление воздуха, пропорциональное квадрату скорости ($F = -\beta V^2$), приводит к уменьшению дальности полета и изменению формы траектории.
- В упрощенной модели (плоская Земля, постоянное $g = 9.81 \text{ м/с}^2$) траектория Ньютона не является ни параболой (из-за сопротивления), ни эллипсом/гиперболой (для этого требуется центральное поле тяготения с переменным g). Это более сложная непараболическая кривая.
- Разница в дальности полета между методами зависит от параметров задачи: начальной скорости, массы тела, площади поперечного сечения и коэффициента сопротивления. При малой массе и большой площади сечения влияние сопротивления воздуха становится значительным.
- Метод Рунге-Кутты 4-го порядка обеспечивает высокую точность численного интегрирования системы дифференциальных уравнений и позволяет получить траектории с достаточной точностью для анализа различий между методами.